Introduction to Software Architecture & Design Thinking

Theory, Edge Cases & Code Examples

- ✓ What is Software Architecture? (Simple Explanation)
- ✓ The Big Question: How Do I Choose an Architecture?
- ✓ The "Perfect Architecture" Trap
- ✓ Conflicting Requirements (The Impossible Trade-off)
- ✓ Legacy System Integration

→ Swipe to learn →

Introduction to Software Architecture & Design Thinking

Theory

📌 1. What is Software Architecture? (Simple Explanation)

Think of architecture like building a house:

House Analogy	Software Analogy	Why It Matters
Foundation	Core data structures & classes	If wrong, everything collapses
Rooms	Components/modules	Each has specific purpose
Doors/Hallways	Interfaces/APIs	How rooms communicate
Plumbing	Data flow	How information moves
Electrical	Control flow	How actions trigger
Blueprint	Architecture diagram	Plan before building

Software Architecture = High-level structure showing:
- What are the major **components**? (e.g., UI, database, business logic)
- How do they **communicate**? (function calls, messages, events)
- What are the **responsibilities**? (who does what)
- What are the **constraints**? (performance, scalability, maintainability)

NOT architecture:
- How you name variables ❌
- Which sorting algorithm you use ❌
- Specific implementation details ❌

IS architecture:
- "We separate UI from business logic" ✅
- "Sensors communicate via event bus" ✅
- "Database layer is independent of UI" ✅

📌 2. The Big Question: How Do I Choose an Architecture?

The 5-Step Thinking Process:

```
Step 1: Understand REQUIREMENTS
    ↓
Step 2: Identify KEY CHARACTERISTICS (what matters most?)
    ↓
Step 3: Analyze CONSTRAINTS (what limits you?)
    ↓
Step 4: Consider TRADE-OFFS (no perfect solution)
    ↓
Step 5: Choose & VALIDATE (test your choice)
```

Let's break each step down with real examples:

Introduction to Software Architecture & Design Thinking

Edge Cases

📌 1. The "Perfect Architecture" Trap (1/2)

Scenario: Spending months designing the "perfect" architecture that handles every possible future requirement.

```
// ❌ OVER-ENGINEERED: Calculator app with microservices architecture

class CalculatorService {
    MessageBroker broker;
    ServiceDiscovery registry;
    LoadBalancer loadBalancer;
    CircuitBreaker circuitBreaker;

public:
    Future<double> add(double a, double b) {
        // Send message to Add microservice
        // Wait for response via message queue
        // Handle retries, timeouts, failures
        // ...100 lines of infrastructure code...
        return broker.sendRequest("add-service", {a, b});
    }
};

// ✅ APPROPRIATE: Calculator app with simple design

class Calculator {
public:
    double add(double a, double b) {
        return a + b;  // That's it!
    }
    double subtract(double a, double b) { return a - b; }
    double multiply(double a, double b) { return a * b; }
    double divide(double a, double b) {
        if (b == 0) throw std::invalid_argument("Division by zero");
        return a / b;
    }
};
```

Introduction to Software Architecture & Design Thinking

Edge Cases

📌 1. The "Perfect Architecture" Trap (2/2)

Why This Happens:

- Fear of future changes
- Resume-driven development
- Not understanding requirements
- Copying enterprise patterns for simple apps

How to Avoid:

- **YAGNI Principle**: You Aren't Gonna Need It
- Start simple, refactor when needed
- Architecture should match current scale, not imagined future scale
- Ask: "What's the simplest design that meets TODAY's requirements?"

The Right Approach:

```
Current needs: 10 users, 4 operations
→ Use simple class

When it grows to: 1000 users, 20 operations, 3 teams
→ Refactor to modules

When it becomes: 1M users, 100 operations, 10 teams
→ Consider microservices
```

📌 2. Conflicting Requirements (The Impossible Trade-off) (1/3)

Scenario: Stakeholders demand mutually exclusive characteristics.

```
Stakeholder demands:
1. "Must be real-time (< 1ms latency)"
2. "Must log every operation for audit"
3. "Must validate every input against database"
4. "Must encrypt all data"
5. "Must run on a $5 microcontroller"

Problem: These requirements are PHYSICALLY IMPOSSIBLE together!
- Database lookup alone takes > 1ms
- Encryption adds latency
- Logging adds latency
- $5 microcontroller can't do all this in 1ms
```

📌 2. Conflicting Requirements (The Impossible Trade-off) (2/3)

How to Handle:

```
// Educate stakeholders with prototype

void demonstrateTradeoffs() {
    std::cout << "Performance test:\n\n";

    // Test 1: Just computation (no logging, validation)
    auto start = now();
    double result = compute(input);
    auto latency1 = now() - start;
    std::cout << "No logging/validation: " << latency1 << "us\n";

    // Test 2: With logging
    start = now();
    log("Computing...");  // File I/O
    result = compute(input);
    log("Done");
    auto latency2 = now() - start;
    std::cout << "With logging: " << latency2 << "us\n";

    // Test 3: With database validation
    start = now();
    if (!database.validate(input)) return;  // Network call
    result = compute(input);
    auto latency3 = now() - start;
    std::cout << "With DB validation: " << latency3 << "us\n";

    // Test 4: Everything
    start = now();
    log("Starting");
    if (!database.validate(input)) return;
    result = encrypt(compute(input));
    log("Done");
    auto latency4 = now() - start;
    std::cout << "With everything: " << latency4 << "us\n";

    std::cout << "\nConclusion: Can't have < 1ms with all features!\n";
    std::cout << "Choose your priority:\n";
    std::cout << "  A) Real-time (< 1ms) but no logging/validation\n";
    std::cout << "  B) Full logging/validation but slower (10-50ms)\n";
}
```

📌 2. Conflicting Requirements (The Impossible Trade-off) (3/3)

Resolution Strategies:

1. **Prototype and show data** (numbers don't lie)
2. **Force prioritization** ("If you could only have 3 out of 5, which 3?")
3. **Propose alternatives** ("Can we log asynchronously? Validate in batch?")
4. **Escalate impossible** ("Need bigger budget or reduce requirements")

📌 3. Legacy System Integration (1/2)

Scenario: Must integrate with a 20-year-old system that has terrible design.

```
// ❌ LEGACY SYSTEM: Global state, no thread safety, cryptic API

// Old system (can't modify)
extern int g_state;  // Global state
extern char g_buffer[256];  // Global buffer (not thread-safe!)

void legacy_init();  // Must call first, no docs on what it does
int legacy_process(char* input, char* output);  // Returns 0 or 1 or -1 (unclear)
void legacy_cleanup();  // Sometimes crashes, no one knows why

// ✅ SOLUTION: Adapter/Wrapper pattern (isolate the ugliness)

class LegacySystemAdapter {
    std::mutex mtx;  // Protect global state
    bool initialized = false;

public:
    LegacySystemAdapter() {
        legacy_init();
        initialized = true;
    }

    ~LegacySystemAdapter() {
        try {
            legacy_cleanup();  // Catch potential crash
        } catch (...) {
            std::cerr << "Legacy cleanup failed (as usual)\n";
        }
    }

    // Clean, modern interface
    std::optional<std::string> process(const std::string& input) {
        std::lock_guard lock(mtx);  // Thread-safe

        if (!initialized) {
            return std::nullopt;
        }

// ...continues on next slide
```

📌 3. Legacy System Integration (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        if (input.size() > 255) {
            return std::nullopt;  // Validate before legacy call
        }

        // Copy to legacy buffer (ugh)
        strcpy(g_buffer, input.c_str());

        int result = legacy_process(g_buffer, g_buffer);

        // Decode cryptic return values
        if (result == 1) {
            return std::string(g_buffer);  // Success
        } else {
            return std::nullopt;  // Failure (0 or -1)
        }
    }
};

// Now rest of code uses clean adapter interface
LegacySystemAdapter adapter;
auto result = adapter.process("input");
if (result) {
    std::cout << "Output: " << *result << "\n";
} else {
    std::cout << "Processing failed\n";
}
```

Key Principles:

- **Isolate ugliness** in adapter layer
- **Translate** legacy API to modern, safe interface
- **Protect** with thread safety, validation
- **Document** undocumented behavior
- **Test extensively** (legacy code often has hidden bugs)

📌 4. Performance vs. Maintainability Dilemma (1/3)

Scenario: Performance-critical code that needs to be maintainable.

```
// Scenario: Real-time sensor processing (< 1ms per frame)

// ❌ OPTION 1: Clean but slow
class SensorProcessor {
public:
    std::vector<Object> process(const std::vector<SensorData>& data) {
        std::vector<Object> objects;

        // Clean, readable, but allocates memory every frame
        for (const auto& sample : data) {
            if (isValid(sample)) {
                Object obj = detectObject(sample);  // Creates object
                if (obj.confidence > 0.8) {
                    objects.push_back(obj);  // Vector grows dynamically
                }
            }
        }

        return objects;  // Copy return (expensive!)
    }
};
// Latency: 5ms (too slow! 5x over budget)

// ❌ OPTION 2: Fast but unmaintainable
Object g_objects[1000];  // Global! Ugh!
int g_count = 0;

void processUgly(SensorData* data, int n) {
    g_count = 0;
    for (int i = 0; i < n; ++i) {
        if (data[i].valid) {
            float conf = /* 20 lines of inlined math */;
            if (conf > 0.8f) {
                g_objects[g_count].x = /* complex calculation */;
                g_objects[g_count].y = /* complex calculation */;
                // ... 50 lines of dense code ...
                g_count++;
            }
// ...continues on next slide
```

📌 4. Performance vs. Maintainability Dilemma (2/3)

```
// ...continued from previous slide
        }
    }
}
// Latency: 0.5ms (fast!)
// Maintainability: Horrible!

// ✅ OPTION 3: Fast AND maintainable (best of both)
class SensorProcessor {
    // Pre-allocated buffers (no runtime allocation)
    std::array<Object, 1000> objectBuffer;
    size_t objectCount = 0;

public:
    // Return view, not copy
    std::span<const Object> process(std::span<const SensorData> data) {
        objectCount = 0;

        for (const auto& sample : data) {
            if (!isValid(sample)) continue;

            Object obj = detectObject(sample);

            if (obj.confidence > 0.8f && objectCount < objectBuffer.size()) {
                objectBuffer[objectCount++] = std::move(obj);
            }
        }

        // Return non-owning view (zero-cost)
        return std::span{objectBuffer.data(), objectCount};
    }

private:
    // Separate, testable functions
    bool isValid(const SensorData& data) const {
        return data.timestamp > 0 && data.quality > 0.5f;
    }

    Object detectObject(const SensorData& data) const {
        // Complex logic in separate function (easier to test)
        Object obj;
// ...continues on next slide
```

📌 4. Performance vs. Maintainability Dilemma (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        obj.x = data.x * calibration.scaleX;
        obj.y = data.y * calibration.scaleY;
        obj.confidence = computeConfidence(data);
        return obj;
    }
};
// Latency: 0.8ms (fast enough!)
// Maintainability: Good!
```

The Middle Ground Strategy:

1. **Pre-allocate** (avoid runtime allocation)
2. **Use modern C++** (std::span, std::array) not raw pointers
3. **Separate concerns** (functions for logic, buffer for performance)
4. **Measure first** (don't optimize prematurely)
5. **Profile** (find real bottlenecks)
6. **Document** why performance tricks used

📌 5. When to Break Your Own Architecture (1/2)

Scenario: Architecture rules conflict with urgent real-world need.

```
// Architecture Rule: "All sensor data goes through event bus"

// Normal flow:
Sensor → Event Bus → Perception → Event Bus → Planning → ...

// Problem: Emergency obstacle detection needs < 5ms, event bus adds 3ms

// ❌ BAD SOLUTION: Ignore the problem, miss emergency obstacle
// ❌ BAD SOLUTION: Bypass architecture randomly, create chaos

// ✅ GOOD SOLUTION: Controlled bypass with clear documentation

class EmergencyObstacleDetector {
    IEventBus* eventBus;
    IPlanner* planner;  // Direct reference (breaks normal architecture)

public:
    EmergencyObstacleDetector(IEventBus* bus, IPlanner* plan)
        : eventBus(bus), planner(plan) {}

    void processLidarData(const LidarData& data) {
        // Normal path: publish to event bus
        eventBus->publish(Event{EventType::Lidar, data});

        // EMERGENCY PATH: Direct call to planner (bypasses architecture)
        if (detectImmediateCollision(data)) {
            // Document WHY we break architecture
            // SAFETY: Direct call for < 5ms latency
            // Bypasses event bus to meet real-time constraint
            planner->emergencyStop();

            // Still publish event for logging/monitoring
            eventBus->publish(Event{EventType::EmergencyStop, {}});
        }
    }

private:
    bool detectImmediateCollision(const LidarData& data) {
// ...continues on next slide
```

📌 5. When to Break Your Own Architecture (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        // Fast collision check
        return data.minDistance < EMERGENCY_THRESHOLD;
    }
};
```

When It's OK to Break Architecture:

- ✅ Safety critical (lives at stake)
- ✅ Performance impossible to meet otherwise
- ✅ Temporary workaround (will refactor later)
- ✅ Clearly documented WHY
- ✅ Isolated (doesn't spread)

When It's NOT OK:

- ❌ "Faster to hack it" (laziness)
- ❌ Deadline pressure (poor planning)
- ❌ Don't understand architecture (learn it)
- ❌ No documentation (future pain)

The Documentation Template:

```
// ARCHITECTURE DEVIATION: Direct call bypasses event bus
// WHY: Emergency detection requires < 5ms latency
// MEASUREMENT: Event bus adds 3ms, direct call adds 0.5ms
// ALTERNATIVE CONSIDERED: Faster event bus (still 2ms, not enough)
// APPROVED BY: Tech lead (date: 2024-03-15)
// TODO: Revisit if event bus optimized to < 1ms
```

📌 6. Over-Abstraction (Too Many Layers) (1/2)

Scenario: Too many abstraction layers make simple tasks complex.

```
// ❌ OVER-ABSTRACTED: 7 layers to read a file!

class IDataSource { virtual std::string read() = 0; };
class IDataProvider { virtual IDataSource* getSource() = 0; };
class IProviderFactory { virtual IDataProvider* create() = 0; };
class IFactoryManager { virtual IProviderFactory* getFactory() = 0; };
class IManagerRegistry { virtual IFactoryManager* getManager() = 0; };
class IRegistryLocator { virtual IManagerRegistry* locate() = 0; };
class ServiceContainer { IRegistryLocator* locator; };

// To read a file, you need:
auto locator = container.getLocator();
auto registry = locator->locate();
auto manager = registry->getManager();
auto factory = manager->getFactory();
auto provider = factory->create();
auto source = provider->getSource();
std::string data = source->read();  // Finally!

// ✅ APPROPRIATE: 2 layers (interface + implementation)

class IDataSource {
public:
    virtual ~IDataSource() = default;
    virtual std::string read() = 0;
};

class FileDataSource : public IDataSource {
    std::string filename;
public:
    FileDataSource(const std::string& file) : filename(file) {}

    std::string read() override {
        std::ifstream file(filename);
        return std::string(std::istreambuf_iterator<char>(file),
                           std::istreambuf_iterator<char>());
    }
};
// ...continues on next slide
```

📌 6. Over-Abstraction (Too Many Layers) (2/2)

[Code continues across slides]

```
// ...continued from previous slide

// Usage:
std::unique_ptr<IDataSource> source = std::make_unique<FileDataSource>("data.txt");
std::string data = source->read();  // Done!
```

The "Maximum 3 Layers" Rule:

For most applications:

```
Layer 1: Interface/Abstract base class
Layer 2: Implementation
Layer 3: (Optional) Adapter/Decorator

If you need Layer 4+, you're probably over-engineering.
```

How to Recognize Over-Abstraction:

- 🚨 "To do X, I need to call 5+ classes"
- 🚨 "I have interfaces with only 1 implementation"
- 🚨 "I have factory factories" (factory for factories)
- 🚨 "My class names are Manager/Handler/Provider/Factory all at once"
- 🚨 "New team members are confused for weeks"

The Simplification Test:

```
Ask: "Can I delete this layer and just call the next one?"
If YES → Delete it!
If NO and you can clearly explain WHY → Keep it.
```

Introduction to Software Architecture & Design Thinking

Code Examples

📌 1. Easy - Single Responsibility (No Architecture Needed) (1/2)

Scenario: Temperature converter (simple utility)

```
#include <iostream>
#include <iomanip>

// No "architecture" needed - just a simple class with functions

class TemperatureConverter {
public:
    static double celsiusToFahrenheit(double celsius) {
        return (celsius * 9.0 / 5.0) + 32.0;
    }

    static double fahrenheitToCelsius(double fahrenheit) {
        return (fahrenheit - 32.0) * 5.0 / 9.0;
    }

    static double celsiusToKelvin(double celsius) {
        return celsius + 273.15;
    }

    static double kelvinToCelsius(double kelvin) {
        return kelvin - 273.15;
    }
};

int main() {
    std::cout << std::fixed << std::setprecision(2);

    double tempC = 25.0;
    std::cout << tempC << "°C = "
              << TemperatureConverter::celsiusToFahrenheit(tempC) << "°F\n";

    double tempF = 77.0;
    std::cout << tempF << "°F = "
              << TemperatureConverter::fahrenheitToCelsius(tempF) << "°C\n";

    return 0;
}

// ...continues on next slide
```

📌 1. Easy - Single Responsibility (No Architecture Needed) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
/*
Output:
25.00°C = 77.00°F
77.00°F = 25.00°C

Lessons:
- Simple problems don't need complex architecture
- Static functions are fine for utilities
- Don't over-engineer!
*/
```

Introduction to Software Architecture & Design Thinking

Code Examples

📌 2. Mid - Layered Architecture (Separation of Concerns) (1/5)

Scenario: Student grade management system

```
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <memory>

// ═══════════════════════════════════════════════════════════
// LAYER 1: Data Layer (How we store data)
// ═══════════════════════════════════════════════════════════

struct Student {
    int id;
    std::string name;
    std::map<std::string, double> grades;  // subject -> grade
};

class IStudentRepository {
public:
    virtual ~IStudentRepository() = default;
    virtual void addStudent(const Student& student) = 0;
    virtual Student* findById(int id) = 0;
    virtual std::vector<Student> getAllStudents() = 0;
};

class InMemoryStudentRepository : public IStudentRepository {
    std::map<int, Student> students;

public:
    void addStudent(const Student& student) override {
        students[student.id] = student;
    }

    Student* findById(int id) override {
        auto it = students.find(id);
        return (it != students.end()) ? &it->second : nullptr;
    }

    std::vector<Student> getAllStudents() override {
// ...continues on next slide
```

📌 2. Mid - Layered Architecture (Separation of Concerns) (2/5)

```
// ...continued from previous slide
        std::vector<Student> result;
        for (const auto& [id, student] : students) {
            result.push_back(student);
        }
        return result;
    }
};

// ═══════════════════════════════════════════════════════════════
// LAYER 2: Business Logic Layer (What we can do with data)
// ═══════════════════════════════════════════════════════════════

class GradeService {
    IStudentRepository* repository;

public:
    GradeService(IStudentRepository* repo) : repository(repo) {}

    void addGrade(int studentId, const std::string& subject, double grade) {
        Student* student = repository->findById(studentId);
        if (!student) {
            throw std::runtime_error("Student not found");
        }

        if (grade < 0 || grade > 100) {
            throw std::invalid_argument("Grade must be 0-100");
        }

        student->grades[subject] = grade;
    }

    double calculateAverage(int studentId) {
        Student* student = repository->findById(studentId);
        if (!student || student->grades.empty()) {
            return 0.0;
        }

        double sum = 0.0;
        for (const auto& [subject, grade] : student->grades) {
            sum += grade;
// ...continues on next slide
```

📌 2. Mid - Layered Architecture (Separation of Concerns) (3/5)

```
// ...continued from previous slide
        }
        return sum / student->grades.size();
    }

    std::string getLetterGrade(double average) {
        if (average >= 90) return "A";
        if (average >= 80) return "B";
        if (average >= 70) return "C";
        if (average >= 60) return "D";
        return "F";
    }
};

// ═══════════════════════════════════════════════════════════════
// LAYER 3: Presentation Layer (How we show data to user)
// ═══════════════════════════════════════════════════════════════

class ConsoleUI {
    GradeService* service;
    IStudentRepository* repository;

public:
    ConsoleUI(GradeService* svc, IStudentRepository* repo)
        : service(svc), repository(repo) {}

    void displayStudentReport(int studentId) {
        Student* student = repository->findById(studentId);
        if (!student) {
            std::cout << "Student not found!\n";
            return;
        }

        std::cout << "═══════════════════════════════════════\n";
        std::cout << "Student Report: " << student->name << "\n";
        std::cout << "═══════════════════════════════════════\n";

        for (const auto& [subject, grade] : student->grades) {
            std::cout << "  " << subject << ": " << grade << "\n";
        }

// ...continues on next slide
```

📌 2. Mid - Layered Architecture (Separation of Concerns) (4/5)

```
// ...continued from previous slide
        double avg = service->calculateAverage(studentId);
        std::string letter = service->getLetterGrade(avg);

        std::cout << "────────────────────────────────────────\n";
        std::cout << "  Average: " << avg << " (" << letter << ")\n";
        std::cout << "═══════════════════════════════════════\n";
    }

    void displayAllStudents() {
        auto students = repository->getAllStudents();

        std::cout << "\nAll Students:\n";
        std::cout << "═══════════════════════════════════════\n";

        for (const auto& student : students) {
            double avg = service->calculateAverage(student.id);
            std::string letter = service->getLetterGrade(avg);

            std::cout << "  [" << student.id << "] "
                      << student.name << " - Average: "
                      << avg << " (" << letter << ")\n";
        }
    }
};

// ═══════════════════════════════════════════════════════════
// MAIN: Wire everything together
// ═══════════════════════════════════════════════════════════

int main() {
    // Create layers
    InMemoryStudentRepository repository;
    GradeService service(&repository);
    ConsoleUI ui(&service, &repository);

    // Add students
    repository.addStudent({1, "Alice", {}});
    repository.addStudent({2, "Bob", {}});

    // Add grades
// ...continues on next slide
```

📌 2. Mid - Layered Architecture (Separation of Concerns) (5/5)

[Code continues across slides]

```
// ...continued from previous slide
    service.addGrade(1, "Math", 95);
    service.addGrade(1, "English", 88);
    service.addGrade(1, "Science", 92);

    service.addGrade(2, "Math", 78);
    service.addGrade(2, "English", 85);
    service.addGrade(2, "Science", 72);

    // Display
    ui.displayStudentReport(1);
    ui.displayStudentReport(2);
    ui.displayAllStudents();

    return 0;
}

/*
Output:
═══════════════════════════════════════
Student Report: Alice
═══════════════════════════════════════
  English: 88
  Math: 95
  Science: 92
───────────────────────────────────────
  Average: 91.6667 (A)
═══════════════════════════════════════
[Similar for Bob...]

Architecture Benefits:
✅ Data Layer: Easy to swap (file storage, database)
✅ Business Logic: Independent of UI and storage
✅ Presentation: Can add web UI without changing logic
✅ Testable: Each layer can be tested independently
*/
```

Introduction to Software Architecture & Design Thinking

Code Examples

📌 3. Mid - Event-Driven Architecture (Decoupling) (1/7)

Scenario: Home automation system (sensors trigger actions)

```
#include <iostream>
#include <vector>
#include <functional>
#include <map>
#include <string>
#include <memory>

// ═══════════════════════════════════════════════════════════
// EVENT SYSTEM (Core of event-driven architecture)
// ═══════════════════════════════════════════════════════════

enum class EventType {
    MotionDetected,
    DoorOpened,
    DoorClosed,
    TemperatureChanged,
    LightToggled
};

struct Event {
    EventType type;
    std::string source;  // Which device
    double value;        // Optional value (temp, light level, etc.)

    Event(EventType t, const std::string& src, double val = 0.0)
        : type(t), source(src), value(val) {}
};

using EventHandler = std::function<void(const Event&)>;

class EventBus {
    std::map<EventType, std::vector<EventHandler>> subscribers;

public:
    void subscribe(EventType type, EventHandler handler) {
        subscribers[type].push_back(handler);
    }

// ...continues on next slide
```

📌 3. Mid - Event-Driven Architecture (Decoupling) (2/7)

```
// ...continued from previous slide
    void publish(const Event& event) {
        std::cout << "[EVENT] " << eventTypeToString(event.type)
                  << " from " << event.source << "\n";

        auto it = subscribers.find(event.type);
        if (it != subscribers.end()) {
            for (const auto& handler : it->second) {
                handler(event);
            }
        }
    }

private:
    std::string eventTypeToString(EventType type) {
        switch (type) {
            case EventType::MotionDetected: return "MotionDetected";
            case EventType::DoorOpened: return "DoorOpened";
            case EventType::DoorClosed: return "DoorClosed";
            case EventType::TemperatureChanged: return "TemperatureChanged";
            case EventType::LightToggled: return "LightToggled";
            default: return "Unknown";
        }
    }
};

// ═══════════════════════════════════════════════════════════════
// DEVICES (Event producers and consumers)
// ═══════════════════════════════════════════════════════════════

class MotionSensor {
    EventBus* bus;
    std::string name;

public:
    MotionSensor(EventBus* eventBus, const std::string& sensorName)
        : bus(eventBus), name(sensorName) {}

    void detectMotion() {
        std::cout << "[" << name << "] Motion detected!\n";
        bus->publish(Event{EventType::MotionDetected, name});
// ...continues on next slide
```

📌 3. Mid - Event-Driven Architecture (Decoupling) (3/7)

```
// ...continued from previous slide
    }
};

class DoorSensor {
    EventBus* bus;
    std::string name;
    bool isOpen = false;

public:
    DoorSensor(EventBus* eventBus, const std::string& sensorName)
        : bus(eventBus), name(sensorName) {}

    void open() {
        if (!isOpen) {
            isOpen = true;
            std::cout << "[" << name << "] Door opened\n";
            bus->publish(Event{EventType::DoorOpened, name});
        }
    }

    void close() {
        if (isOpen) {
            isOpen = false;
            std::cout << "[" << name << "] Door closed\n";
            bus->publish(Event{EventType::DoorClosed, name});
        }
    }
};

class Light {
    std::string name;
    bool isOn = false;

public:
    Light(const std::string& lightName) : name(lightName) {}

    void turnOn() {
        if (!isOn) {
            isOn = true;
            std::cout << "  → [" << name << "] Light turned ON 💡\n";
// ...continues on next slide
```

📌 3. Mid - Event-Driven Architecture (Decoupling) (4/7)

```
// ...continued from previous slide
        }
    }

    void turnOff() {
        if (isOn) {
            isOn = false;
            std::cout << "  → [" << name << "] Light turned OFF\n";
        }
    }
};

class Alarm {
    std::string name;
    bool isActive = false;

public:
    Alarm(const std::string& alarmName) : name(alarmName) {}

    void activate() {
        isActive = true;
        std::cout << "  → [" << name << "] 🚨 ALARM ACTIVATED! 🚨\n";
    }

    void deactivate() {
        isActive = false;
        std::cout << "  → [" << name << "] Alarm deactivated\n";
    }
};

// ═══════════════════════════════════════════════════════════
// AUTOMATION RULES (Event handlers)
// ═══════════════════════════════════════════════════════════

class HomeAutomation {
    Light livingRoomLight;
    Light hallwayLight;
    Alarm securityAlarm;
    bool homeOccupied = false;

public:
// ...continues on next slide
```

📌 3. Mid - Event-Driven Architecture (Decoupling) (5/7)

```
// ...continued from previous slide
    HomeAutomation(EventBus* bus)
        : livingRoomLight("Living Room")
        , hallwayLight("Hallway")
        , securityAlarm("Security System") {

        // Subscribe to events

        // Rule 1: Motion detected → Turn on nearby light
        bus->subscribe(EventType::MotionDetected, [this](const Event& e) {
            std::cout << "  [Rule] Motion detected, turning on lights\n";
            if (e.source == "Living Room Sensor") {
                livingRoomLight.turnOn();
            } else if (e.source == "Hallway Sensor") {
                hallwayLight.turnOn();
            }
        });

        // Rule 2: Door opened → Turn on hallway light + deactivate alarm
        bus->subscribe(EventType::DoorOpened, [this](const Event& e) {
            std::cout << "  [Rule] Door opened, welcome home!\n";
            hallwayLight.turnOn();
            securityAlarm.deactivate();
            homeOccupied = true;
        });

        // Rule 3: Door closed → Activate alarm if leaving
        bus->subscribe(EventType::DoorClosed, [this](const Event& e) {
            if (homeOccupied) {
                std::cout << "  [Rule] Door closed while home, just a door close\n";
            } else {
                std::cout << "  [Rule] Door closed, activating security\n";
                securityAlarm.activate();
            }
        });

        // Rule 4: Motion when alarm active → Trigger alarm
        bus->subscribe(EventType::MotionDetected, [this](const Event& e) {
            if (!homeOccupied) {
                std::cout << "  [Rule] Motion while away - INTRUDER!\n";
                securityAlarm.activate();
// ...continues on next slide
```

📌 3. Mid - Event-Driven Architecture (Decoupling) (6/7)

```
// ...continued from previous slide
            }
        });
    }
};

// ═══════════════════════════════════════════════════════════════
// MAIN: Simulate home automation
// ═══════════════════════════════════════════════════════════════

int main() {
    std::cout << "Home Automation System (Event-Driven Architecture)\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // Create event bus (central communication)
    EventBus eventBus;

    // Create automation rules
    HomeAutomation automation(&eventBus);

    // Create sensors
    MotionSensor livingRoomSensor(&eventBus, "Living Room Sensor");
    MotionSensor hallwaySensor(&eventBus, "Hallway Sensor");
    DoorSensor frontDoor(&eventBus, "Front Door");

    // Scenario 1: Arriving home
    std::cout << "═══ Scenario 1: Arriving Home ═══\n";
    frontDoor.open();
    hallwaySensor.detectMotion();
    livingRoomSensor.detectMotion();
    std::cout << "\n";

    // Scenario 2: Leaving home
    std::cout << "═══ Scenario 2: Leaving Home ═══\n";
    frontDoor.open();
    frontDoor.close();
    std::cout << "\n";

    // Scenario 3: Intruder!
    std::cout << "═══ Scenario 3: Motion While Away (Intruder!) ═══\n";
    livingRoomSensor.detectMotion();
// ...continues on next slide
```

📌 3. Mid - Event-Driven Architecture (Decoupling) (7/7)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "\n";

    return 0;
}

/*
Output:
Home Automation System (Event-Driven Architecture)
═══════════════════════════════════════════════════

═══ Scenario 1: Arriving Home ═══
[Front Door] Door opened
[EVENT] DoorOpened from Front Door
  [Rule] Door opened, welcome home!
  → [Hallway] Light turned ON 💡
  → [Security System] Alarm deactivated
[Hallway Sensor] Motion detected!
[EVENT] MotionDetected from Hallway Sensor
  [Rule] Motion detected, turning on lights
[Living Room Sensor] Motion detected!
[EVENT] MotionDetected from Living Room Sensor
  [Rule] Motion detected, turning on lights
  → [Living Room] Light turned ON 💡

[Similar for other scenarios...]

Architecture Benefits:
✅ Decoupled: Sensors don't know about lights/alarms
✅ Flexible: Easy to add new rules without changing sensors
✅ Scalable: Can add 100 sensors without complexity
✅ Testable: Can test rules by publishing events
✅ Asynchronous: Can process events in any order
*/
```

📌 4. Advanced - Component-Based Architecture (ECS Pattern) (1/7)

Scenario: Simple 2D game with Entity-Component-System

```
#include <iostream>
#include <vector>
#include <memory>
#include <unordered_map>
#include <string>
#include <cmath>

// ═══════════════════════════════════════════════════════════
// COMPONENTS (Pure data, no logic)
// ═══════════════════════════════════════════════════════════

struct Position {
    double x, y;
};

struct Velocity {
    double dx, dy;
};

struct Health {
    int current, max;
};

struct Renderable {
    std::string sprite;
    int zOrder;  // Draw order
};

// ═══════════════════════════════════════════════════════════
// ENTITY (Just an ID + components)
// ═══════════════════════════════════════════════════════════

using EntityID = int;

class Entity {
    static EntityID nextID;
    EntityID id;

// ...continues on next slide
```

📌 4. Advanced - Component-Based Architecture (ECS Pattern) (2/7)

```
// ...continued from previous slide
public:
    Entity() : id(nextID++) {}
    EntityID getID() const { return id; }
};

EntityID Entity::nextID = 0;

// ═══════════════════════════════════════════════════════════
// COMPONENT MANAGER (Store components separately)
// ═══════════════════════════════════════════════════════════

template<typename T>
class ComponentManager {
    std::unordered_map<EntityID, T> components;

public:
    void add(EntityID entity, const T& component) {
        components[entity] = component;
    }

    T* get(EntityID entity) {
        auto it = components.find(entity);
        return (it != components.end()) ? &it->second : nullptr;
    }

    bool has(EntityID entity) {
        return components.find(entity) != components.end();
    }

    void remove(EntityID entity) {
        components.erase(entity);
    }

    std::unordered_map<EntityID, T>& getAll() {
        return components;
    }
};

// ═══════════════════════════════════════════════════════════
// SYSTEMS (Logic that operates on components)
// ...continues on next slide
```

📌 4. Advanced - Component-Based Architecture (ECS Pattern) (3/7)

```
// ...continued from previous slide
// ═══════════════════════════════════════════════════════════

class MovementSystem {
    ComponentManager<Position>* positions;
    ComponentManager<Velocity>* velocities;

public:
    MovementSystem(ComponentManager<Position>* pos,
                   ComponentManager<Velocity>* vel)
        : positions(pos), velocities(vel) {}

    void update(double deltaTime) {
        // Process all entities that have BOTH position AND velocity
        for (auto& [entityID, velocity] : velocities->getAll()) {
            Position* pos = positions->get(entityID);
            if (pos) {
                pos->x += velocity.dx * deltaTime;
                pos->y += velocity.dy * deltaTime;
            }
        }
    }
};

class RenderSystem {
    ComponentManager<Position>* positions;
    ComponentManager<Renderable>* renderables;

public:
    RenderSystem(ComponentManager<Position>* pos,
                 ComponentManager<Renderable>* rend)
        : positions(pos), renderables(rend) {}

    void draw() {
        std::cout << "\n═══ Frame Render ═══\n";

        // Draw all entities that have BOTH position AND renderable
        for (auto& [entityID, renderable] : renderables->getAll()) {
            Position* pos = positions->get(entityID);
            if (pos) {
                std::cout << "[" << renderable.sprite << "] at ("
// ...continues on next slide
```

📌 4. Advanced - Component-Based Architecture (ECS Pattern) (4/7)

```
// ...continued from previous slide
                          << pos->x << ", " << pos->y << ")\n";
            }
        }
    }
};

class HealthSystem {
    ComponentManager<Health>* healths;

public:
    HealthSystem(ComponentManager<Health>* hp) : healths(hp) {}

    void takeDamage(EntityID entity, int damage) {
        Health* health = healths->get(entity);
        if (health) {
            health->current -= damage;
            if (health->current < 0) health->current = 0;

            std::cout << "[Entity " << entity << "] took " << damage
                      << " damage! HP: " << health->current << "/"
                      << health->max;

            if (health->current == 0) {
                std::cout << " (DEAD)";
            }
            std::cout << "\n";
        }
    }

    void heal(EntityID entity, int amount) {
        Health* health = healths->get(entity);
        if (health) {
            health->current += amount;
            if (health->current > health->max) {
                health->current = health->max;
            }

            std::cout << "[Entity " << entity << "] healed " << amount
                      << "! HP: " << health->current << "/" << health->max << "\n";
        }
// ...continues on next slide
```

📌 4. Advanced - Component-Based Architecture (ECS Pattern) (5/7)

```
// ...continued from previous slide
    }
};

// ═══════════════════════════════════════════════════════════════
// MAIN: Create game entities
// ═══════════════════════════════════════════════════════════════

int main() {
    std::cout << "Component-Based Architecture (ECS Pattern)\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // Create component managers
    ComponentManager<Position> positions;
    ComponentManager<Velocity> velocities;
    ComponentManager<Health> healths;
    ComponentManager<Renderable> renderables;

    // Create systems
    MovementSystem movementSystem(&positions, &velocities);
    RenderSystem renderSystem(&positions, &renderables);
    HealthSystem healthSystem(&healths);

    // Create entities
    Entity player;
    Entity enemy;
    Entity tree;  // Static decoration

    std::cout << "Creating entities...\n";
    std::cout << "  Player ID: " << player.getID() << "\n";
    std::cout << "  Enemy ID: " << enemy.getID() << "\n";
    std::cout << "  Tree ID: " << tree.getID() << "\n\n";

    // Add components to player (movable, damageable, renderable)
    positions.add(player.getID(), {0.0, 0.0});
    velocities.add(player.getID(), {10.0, 5.0});
    healths.add(player.getID(), {100, 100});
    renderables.add(player.getID(), {"🧙 Player", 10});

    // Add components to enemy (movable, damageable, renderable)
    positions.add(enemy.getID(), {50.0, 50.0});
// ...continues on next slide
```

📌 4. Advanced - Component-Based Architecture (ECS Pattern) (6/7)

```
// ...continued from previous slide
    velocities.add(enemy.getID(), {-5.0, 3.0});
    healths.add(enemy.getID(), {50, 50});
    renderables.add(enemy.getID(), {"👹 Enemy", 10});

    // Add components to tree (only position + renderable, NOT movable)
    positions.add(tree.getID(), {100.0, 100.0});
    renderables.add(tree.getID(), {"🌳 Tree", 5});
    // Note: No velocity → tree won't move!

    // Game loop simulation
    for (int frame = 0; frame < 3; ++frame) {
        std::cout << "\n═══════════ Frame " << frame << " ═══════════\n";

        // Update systems
        movementSystem.update(0.1);  // deltaTime = 0.1 seconds

        // Render
        renderSystem.draw();
    }

    // Combat simulation
    std::cout << "\n═══════════ Combat ═══════════\n";
    healthSystem.takeDamage(enemy.getID(), 30);
    healthSystem.takeDamage(player.getID(), 20);
    healthSystem.heal(player.getID(), 15);
    healthSystem.takeDamage(enemy.getID(), 25);  // Kill enemy

    return 0;
}

/*
Output:
Component-Based Architecture (ECS Pattern)
═══════════════════════════════════════════════════════

Creating entities...
  Player ID: 0
  Enemy ID: 1
  Tree ID: 2

// ...continues on next slide
```

📌 4. Advanced - Component-Based Architecture (ECS Pattern) (7/7)

[Code continues across slides]

```
// ...continued from previous slide
═══════════ Frame 0 ═══════════
═══ Frame Render ═══
[🧙 Player] at (0, 0)
[👹 Enemy] at (50, 50)
[🌳 Tree] at (100, 100)

═══════════ Frame 1 ═══════════
═══ Frame Render ═══
[🧙 Player] at (1, 0.5)
[👹 Enemy] at (49.5, 50.3)
[🌳 Tree] at (100, 100)  ← DOESN'T MOVE (no velocity)

[Similar for Frame 2...]

═══════════ Combat ═══════════
[Entity 1] took 30 damage! HP: 20/50
[Entity 0] took 20 damage! HP: 80/100
[Entity 0] healed 15! HP: 95/100
[Entity 1] took 25 damage! HP: 0/50 (DEAD)

Architecture Benefits:
✅ Data-oriented: Components stored contiguously (cache-friendly)
✅ Flexible composition: Easy to create new entity types
✅ Reusable systems: Same system works on all entities
✅ Performance: Systems process components in tight loops
✅ Easy to add features: New component + system, no changes to entities
*/
```

What You Learned

Key Takeaways

- ✓ What is Software Architecture? (Simple Explanation)
- ✓ The Big Question: How Do I Choose an Architecture?
- ✓ The "Perfect Architecture" Trap
- ✓ Conflicting Requirements (The Impossible Trade-off)

→ Swipe to learn →